

Homework 6

Damion Huppert

```
In [2]: using Pkg
        Pkg.add("CSV")
        using CSV
        Pkg.add("DataFrames")
        using DataFrames
        Pkg.add("JuMP")
        using JuMP
        Pkg.add("HiGHS")
        using HiGHS
        Pkg.add("Plots")
        using Plots
        Pkg.add("Gurobi")
        using Gurobi
        Pkg.add("LinearAlgebra")
        using LinearAlgebra
        Pkg.add("NamedArrays")
        using NamedArrays
```


$\min m$

Constraints:

$$\begin{aligned} \sum_{i \in I} x_{ji} &= 1 \quad \forall j \in J && \text{Each job is assigned to one station} \\ \sum_{i \in I} z_i^{T1} &= 1 && \text{Task 1 is assigned to one station} \\ \sum_{j \in J} p_{ji} x_{ji} + U_i y_i &\leq T_i \quad \forall i \in I && \text{Do not use more time at the station} \\ \sum_{j \in J} p_{ji} x_{ji} + U_i y_i &\leq m \quad \forall i \in I && \text{Makespan is larger than all station times} \\ x_{ji} &\leq y_i && \forall j \in J, i \in I \\ x_{ji} &\in \{0, 1\}, y_i \in \{0, 1\}, z_i^{T1} \in \{0, 1\}, m \in \mathbb{R} \end{aligned}$$

Question 1-2

```
In [30]: # constants
# Sets
I = 1:4                                # Stations
J = 1:8                                # Jobs
T1_jobs = 1:3                          # Task 1 jobs

# Parameters
p = [
    [17, 18, 12, 16], # Task1-Step1
    [20, 18, 17, 14], # Task1-Step2
    [17, 21, 9, 21],  # Task1-Step3
    [13, 18, 21, 12], # Task2-Step1
    [19, 21, 17, 15], # Task2-Step2
    [11, 13, 9, 14],  # Task3-Step1
    [13, 9, 9, 12],   # Task3-Step2
    [9, 13, 17, 17]   # Task3-Step3
]
# p[j][i] = time of job j at station i

T_avail = [41, 45, 50, 46] # Station time available
U = [10, 12, 8, 6]         # Setup times

model = Model(Gurobi.Optimizer)

@variable(model, x[j in J, i in I], Bin)
@variable(model, y[i in I], Bin)
@variable(model, zT1[i in I], Bin)
@variable(model, m >= 0)

@objective(model, Min, m)

@constraint(model, [j in J], sum(x[j, i] for i in I) == 1) # Each job is assigned to one station
```

```

@constraint(model, sum(zT1[i] for i in I) == 1) # Only one station for T1 jobs
@constraint(model, [i in I, j in T1_jobs], x[j, i] == zT1[i]) # T1 jobs assigned to station
@constraint(model, [i in I], sum(p[j][i] * x[j, i] for j in J) + U[i] * y[i] == 1)
@constraint(model, [i in I], sum(p[j][i] * x[j, i] for j in J) + U[i] * y[i] == 1)
@constraint(model, [j in J, i in I], x[j, i] <= y[i]) # Job can only be assigned to station if used

set_silent(model)
optimize!(model)

println("Optimal Time: ", value(m))
for i in I
    if value(y[i]) > 0.5
        println("Station $i is used:")
        for j in J
            if value(x[j, i]) > 0.5
                println("  Job $j assigned to Station $i")
            end
        end
    end
end
end

```

Set parameter Username

Set parameter LicenseID to value 2649580

Academic license – for non-commercial use only – expires 2026-04-09

Optimal Time: 46.0

Station 1 is used:

Job 6 assigned to Station 1

Job 7 assigned to Station 1

Station 2 is used:

Job 4 assigned to Station 2

Job 8 assigned to Station 2

Station 3 is used:

Job 1 assigned to Station 3

Job 2 assigned to Station 3

Job 3 assigned to Station 3

Station 4 is used:

Job 5 assigned to Station 4

Question 2-1

Sets:

- $I = \{1, 2, 3\}$: Boilers
- $J = \{1, 2, 3\}$: Turbines

Decision Variables:

x_i tons of steam produced by boiler i

$y_i \in \{0, 1\}$ 1 if boiler i is used, 0 otherwise

t_j tons of steam processed by turbine j

$$z_j \in \{0, 1\} \quad 1 \text{ if turbine } j \text{ is used, } 0 \text{ otherwise}$$

Parameters:

c_i cost per ton produced from boiler i

q_j cost per ton proccessed from turbine j

p_j power produced per ton of steam

Objective:

$$\min 10x_1 + 8x_2 + 7x_3 + 2t_1 + 3t_2 + 4t_3$$

Subject to:

$$2t_1 + 3t_2 + 4t_3 \geq 8000$$

$$400y_1 \leq x_1 \leq 1000y_1$$

$$200y_2 \leq x_2 \leq 900y_2$$

$$300y_3 \leq x_3 \leq 800y_3$$

$$300z_1 \leq t_1 \leq 600z_1$$

$$500z_2 \leq t_2 \leq 800z_2$$

$$600z_3 \leq t_3 \leq 900z_3$$

$$t_1 + t_2 + t_3 \leq x_1 + x_2 + x_3$$

$$x_i \geq 0$$

$$t_l \geq 0$$

$$y_i \text{ binary}$$

$$z_i \text{ binary}$$

Question 2-2

$$\text{IF } y_1 - 1 == 0 \text{ THEN } t_1 + t_3 \geq 750$$

$$t_1 + t_3 - 750 \geq m(y_1 - 1)$$

$$t_1 + t_3 \geq 750y_1$$

Question 2-3

$$\text{IF } (y_1 + y_2 + y_3) - 3 == 0 \text{ THEN } z_1 + z_3 \leq 1$$

$$z_1 + z_3 - 1 \leq M((y_1 + y_2 + y_3) - 3)$$

Question 2-4

IF $(z_2) - 1 == 0$ THEN $20x_1 + 30x_2 + 35x_3 \leq 5000$

$$20x_1 + 30x_2 + 35x_3 - 5000 \leq M((z_2) - 1)$$

Question 3-1

Decision variables:

$x_i \in \{0, 1\}$ 1 if district i is an auror location 0 otherwise

$d_i \in \{0, 1\}$ 1 if district i is within 2 seconds of an auror

Objective:

$$\max \sum_{i \in I} d_i \cdot p_i$$

Subject to:

$$\sum_{i \in I} x_i = 3$$

$$d_j \leq \sum_{i \in I: t_{ij} \leq 2} x_i \quad \forall j \in I$$

$$x_i, d_j \in \{0, 1\} \quad \forall i, j \in I$$

Question 3-2

```
In [31]: using JuMP
using Gurobi

districts = 1:8
p = [40, 30, 35, 20, 15, 50, 45, 60]
t = [
    [0, 3, 4, 6, 1, 9, 8, 10],
    [3, 0, 5, 4, 8, 6, 1, 9],
    [4, 5, 0, 2, 2, 3, 5, 7],
    [6, 4, 2, 0, 3, 2, 5, 4],
    [1, 8, 2, 3, 0, 2, 2, 4],
    [9, 6, 3, 2, 2, 0, 3, 2],
    [8, 1, 5, 5, 2, 3, 0, 2],
    [10, 9, 7, 4, 4, 2, 2, 0]
]

model = Model(Gurobi.Optimizer)
```

```

# Variables
@variable(model, x[districts], Bin) # 1 if auror is placed in district i
@variable(model, d[districts], Bin) # 1 if district i is covered

# Objective: Maximize total covered population
@objective(model, Max, sum(p[i] * d[i] for i in districts))

# Constraint: exactly 3 aurors
@constraint(model, sum(x[i] for i in districts) == 3)

# Coverage constraint
for j in districts
    @constraint(model, d[j] <= sum(x[i] for i in districts if t[i][j] <= 2))
end

# Solve
optimize!(model)

# Output
println("Optimal covered population: ", objective_value(model))
println("Auror locations (x): ", [i for i in districts if value(x[i]) > 0.5])
println("Covered districts (d): ", [i for i in districts if value(d[i]) > 0.5])

```

Set parameter Username
Set parameter LicenseID to value 2649580
Academic license – for non-commercial use only – expires 2026-04-09
Gurobi Optimizer version 12.0.1 build v12.0.1rc0 (mac64[arm] – Darwin 24.0.0 24A335)

CPU model: Apple M1
Thread count: 8 physical cores, 8 logical processors, using up to 8 threads

Optimize a model with 9 rows, 16 columns and 42 nonzeros
Model fingerprint: 0xa206f7f0
Variable types: 0 continuous, 16 integer (16 binary)
Coefficient statistics:

Matrix range [1e+00, 1e+00]
Objective range [2e+01, 6e+01]
Bounds range [0e+00, 0e+00]
RHS range [3e+00, 3e+00]

Found heuristic solution: objective 295.0000000
Presolve removed 9 rows and 16 columns
Presolve time: 0.00s
Presolve: All rows and columns removed

Explored 0 nodes (0 simplex iterations) in 0.00 seconds (0.00 work units)
Thread count was 1 (of 8 available processors)

Solution count 1: 295

Optimal solution found (tolerance 1.00e-04)
Best objective 2.950000000000e+02, best bound 2.950000000000e+02, gap 0.000 0%

User-callback calls 338, time in user-callback 0.00 sec
Optimal covered population: 295.0
Auror locations (x): [5, 6, 7]
Covered districts (d): [1, 2, 3, 4, 5, 6, 7, 8]

Question 4-1

Decision Variables:

c_{ij} Cost of switching from task i to task j

x_{ij} 1 if we switch from task i to task j 0 otherwise

Objective:

$$\min \sum_{i \in I} \sum_{j \in I} c_{ij} x_{ij} + p_i x_{ij}$$

Subject to:

$$\sum_{i \in I} x_{ij} = 1 \quad \forall j \in I$$

$$\sum_{j \in I} x_{ij} = 1 \quad \forall i \in I$$

$$x_{ii} = 0 \quad \forall i \in I$$

Question 4-2

Constraint to remove subcontours

$$u_i - u_j + nx_{ij} \leq n - 1 \quad \forall i, \forall j \neq i$$

Question 4-3

```
In [32]: # Job identifiers
job_pairs = [(:1, :1), (:1, :2), (:1, :3),
              (:2, :1), (:2, :2),
              (:3, :1), (:3, :2), (:3, :3)]
n = length(job_pairs)

# Processing times
make_times = Dict{zip(job_pairs, [10, 13, 7, 12, 11, 8, 8, 9])}

# Cleaning durations matrix
clean_duration_matrix = [
    0 5 9 20 14 14 10 8;
    19 0 10 20 9 6 13 6;
    12 13 0 18 19 20 20 11;
    20 21 22 0 25 19 20 25;
    7 11 18 9 0 9 9 15;
    12 10 16 5 16 0 9 7;
    14 16 11 5 14 15 0 11;
    10 11 7 14 8 19 16 0
]
duration_NA = NamedArray(clean_duration_matrix, (job_pairs, job_pairs), ("j", "c"))

# Create model
model = Model(Gurobi.Optimizer)

@variable(model, x[job_pairs, job_pairs], Bin)
@variable(model, 2 <= u[2:n] <= n)

# Objective: Minimize total processing + cleaning time
@objective(model, Min,
    sum((make_times[i] + duration_NA[i, j]) * x[i, j]
        for i in job_pairs, j in job_pairs if i != j)
    )
```

```

# Each job is entered once
@constraint(model, [j in job_pairs], sum(x[i, j] for i in job_pairs if i !=

# Each job is exited once
@constraint(model, [i in job_pairs], sum(x[i, j] for j in job_pairs if i !=

# MTZ subtour elimination (indexing u from 1:n)
@constraint(model, [i in 2:n, j in 2:n; i != j],
    u[i] - u[j] + n * x[job_pairs[i], job_pairs[j]] <= n - 1
)

# Solve
optimize!(model)

# Extract tour order
order = [job_pairs[1]]
current_job = job_pairs[1]
for _ in 1:n-1
    for j in job_pairs
        if current_job != j && value(x[current_job, j]) > 0.5
            push!(order, j)
            current_job = j
            break
        end
    end
end

# Calculate total time
total_time = 0
for i in 1:n
    j = order[mod1(i+1, n)]
    total_time += make_times[order[i]] + duration_NA[order[i], j]
end

# Output
println("Optimal job order:")
for job in order
    println(job)
end
println("Total time (processing + cleaning): ", total_time)

```

Set parameter Username

Set parameter LicenseID to value 2649580

Academic license – for non-commercial use only – expires 2026-04-09

Gurobi Optimizer version 12.0.1 build v12.0.1rc0 (mac64[arm] – Darwin 24.0.0 24A335)

CPU model: Apple M1

Thread count: 8 physical cores, 8 logical processors, using up to 8 threads

Optimize a model with 58 rows, 71 columns and 238 nonzeros

Model fingerprint: 0x683e9fd4

Variable types: 7 continuous, 64 integer (64 binary)

Coefficient statistics:

Matrix range [1e+00, 8e+00]
 Objective range [1e+01, 4e+01]
 Bounds range [2e+00, 8e+00]
 RHS range [1e+00, 7e+00]
 Found heuristic solution: objective 173.0000000
 Presolve removed 0 rows and 8 columns
 Presolve time: 0.00s
 Presolved: 58 rows, 63 columns, 448 nonzeros
 Variable types: 7 continuous, 56 integer (56 binary)

 Root relaxation: objective 1.503333e+02, 21 iterations, 0.00 seconds (0.00 work units)

Nodes		Current Node				Objective Bounds			Work	
Expl	Unexpl	Obj	Depth	IntInf		Incumbent	BestBd	Gap	It/Node	Time
	0	0	150.33333	0	10	173.00000	150.33333	13.1%	–	0s
H	0	0				167.0000000	150.33333	10.0%	–	0s
H	0	0				151.0000000	150.33333	0.44%	–	0s
	0	0	150.33333	0	10	151.00000	150.33333	0.44%	–	0s

Explored 1 nodes (25 simplex iterations) in 0.00 seconds (0.00 work units)
 Thread count was 8 (of 8 available processors)

Solution count 3: 151 167 173

Optimal solution found (tolerance 1.00e-04)
 Best objective 1.510000000000e+02, best bound 1.510000000000e+02, gap 0.0000%

User-callback calls 240, time in user-callback 0.00 sec

Optimal job order:

(1, 1)
 (1, 2)
 (2, 2)
 (3, 2)
 (2, 1)
 (3, 1)
 (3, 3)
 (1, 3)

Total time (processing + cleaning): 151

Question 5-1

- I = set of locations
- 90 Uber cars
- 10 locations
- d_{ij} = Euclidean distance between locations i and j

Decision variables:

x_{ij}^k number of cars transferred from i to j in tier k

Objective

$$\min \sum_{i \in I} \sum_{j \in I} (5d_{ij}x_{ij}^1 + 3d_{ij}x_{ij}^2 + 1d_{ij}x_{ij}^3)$$

Subject To:

$$\text{current}_i + \sum_{j \in I} (x_{ji}^1 + x_{ji}^2 + x_{ji}^3) - \sum_{j \in I} (x_{ij}^1 + x_{ij}^2 + x_{ij}^3) = \text{required}_i \quad \text{Cars at loc } i$$

$$0 \leq x_{ij}^1 \leq 2$$

$$0 \leq x_{ij}^2 \leq 3$$

$$0 \leq x_{ij}^3 \leq 2$$

$$x_{ij}^1 + x_{ij}^2 + x_{ij}^3 \leq 8 \quad \text{At most 8 cars can drive from one loc to another}$$

Question 5-2

```
In [ ]: locations = [
    ("Capitol Square", 0, 0, 8, 6),
    ("Epic Systems", 20, 20, 6, 11),
    ("Overture Center", 18, 10, 8, 4),
    ("Camp Randall", 30, 12, 9, 8),
    ("Hilldale Mall", 35, 0, 9, 12),
    ("West Towne Mall", 33, 25, 7, 2),
    ("Olbrich Gardens", 4, 27, 13, 14),
    ("The Arboretum", 4, 10, 7, 11),
    ("Picnic Point", 11, 0, 9, 15),
    ("Garver Feed Mill", 2, 15, 14, 7)
]

n = length(locations)
names = [loc[1] for loc in locations]
coords = [(loc[2], loc[3]) for loc in locations]
required = [loc[4] for loc in locations]
current = [loc[5] for loc in locations]
d = [sqrt((coords[i][1] - coords[j][1])**2 + (coords[i][2] - coords[j][2])**2)

model = Model(HiGHS.Optimizer)

@variable(model, 0 <= x[1:n, 1:n, 1:3], Int)

@objective(model, Min,
    sum(5 * d[i,j] * x[i,j,1] + 3 * d[i,j] * x[i,j,2] + 1 * x[i,j,3] for i in 1:n, j in 1:n)
)
```

```

for i in 1:n
    @constraint(model,
        current[i]
        + sum(x[j,i,1] + x[j,i,2] + x[j,i,3] for j in 1:n if j != i)
        - sum(x[i,j,1] + x[i,j,2] + x[i,j,3] for j in 1:n if j != i)
        == required[i]
    )
end

for i in 1:n, j in 1:n
    if i != j
        @constraint(model, x[i,j,1] <= 2)
        @constraint(model, x[i,j,2] <= 3)
        @constraint(model, x[i,j,3] <= 2)
    else
        @constraint(model, x[i,j,1] == 0)
        @constraint(model, x[i,j,2] == 0)
        @constraint(model, x[i,j,3] == 0)
    end
end

set_silent(model)
optimize!(model)

total_cost = objective_value(model)
println("Total transport cost: \$", total_cost)

for i in 1:n
    for j in 1:n
        for k in 1:3
            if value(x[i,j,k]) > 0.5
                println("Move ", value(x[i,j,k]), " car(s) from ", names[i],
                    end
            end
        end
    end
end
end

```

Total transport cost: \$19.0

Move 2.0 car(s) from Epic Systems to Capitol Square in tier 3
 Move 1.0 car(s) from Epic Systems to West Towne Mall in tier 3
 Move 2.0 car(s) from Epic Systems to Garver Feed Mill in tier 3
 Move 1.0 car(s) from Hilldale Mall to Overture Center in tier 3
 Move 2.0 car(s) from Hilldale Mall to West Towne Mall in tier 3
 Move 1.0 car(s) from Olbrich Gardens to Garver Feed Mill in tier 3
 Move 2.0 car(s) from The Arboretum to Overture Center in tier 3
 Move 2.0 car(s) from The Arboretum to Garver Feed Mill in tier 3
 Move 1.0 car(s) from Picnic Point to Overture Center in tier 3
 Move 1.0 car(s) from Picnic Point to Camp Randall in tier 3
 Move 2.0 car(s) from Picnic Point to West Towne Mall in tier 3
 Move 2.0 car(s) from Picnic Point to Garver Feed Mill in tier 3

Question 6

$H = \{1, 2, 3, 4, 5\}$ Houses 1-5
 $C = \{1, 2, 3, 4, 5\}$ Colors 1-5
 $N = \{1, 2, 3, 4, 5\}$ Nationalities 1-5
 $D = \{1, 2, 3, 4, 5\}$ Drinks 1-5
 $T = \{1, 2, 3, 4, 5\}$ Cigs 1-5
 $P = \{1, 2, 3, 4, 5\}$ Pets 1-5

Variables:

$color_{h,c}$ 1 if house h is color c
 $nationality_{h,n}$ 1 if house h has nationality n
 $drink_{h,c}$ 1 if house h drinks d
 $cigar_{h,t}$ 1 if house h smokes t
 $pet_{h,p}$ 1 if house h has pet p

Constraints:

$\sum_{c \in C} color_{h,c} = 1$ houses can only be one color
 $\sum_{n \in N} nationality_{h,n} = 1$ houses can only be one nationality
 $\sum_{c \in C} drink_{h,c} = 1$ houses can only have one drink
 $\sum_{t \in T} cigar_{h,t} = 1$ houses can only have one cigar
 $\sum_{p \in P} pet_{h,p} = 1$ houses can only have one pet

Clues: (I will document in the code)

In [9]: **using** Printf

```

houses = 1:5
colors = [:Red, :Green, :White, :Yellow, :Blue]
nationalities = [:Brit, :Swede, :Dane, :Norwegian, :German]
drinks = [:Tea, :Coffee, :Milk, :Beer, :Water]
cigars = [:PallMall, :Dunhill, :Blends, :BlueMaster, :Prince]
pets = [:Dogs, :Birds, :Cats, :Horses, :Fish]

model = Model(HiGHS.Optimizer)

@variable(model, color[h in houses, c in colors], Bin)

```

```

@variable(model, nationality[h in houses, n in nationalities], Bin)
@variable(model, drink[h in houses, d in drinks], Bin)
@variable(model, cigar[h in houses, g in cigars], Bin)
@variable(model, pet[h in houses, p in pets], Bin)

# Assignment constraints: each house has exactly one of each attribute
for h in houses
    @constraint(model, sum(color[h, c] for c in colors) == 1)
    @constraint(model, sum(nationality[h, n] for n in nationalities) == 1)
    @constraint(model, sum(drink[h, d] for d in drinks) == 1)
    @constraint(model, sum(cigar[h, g] for g in cigars) == 1)
    @constraint(model, sum(pet[h, p] for p in pets) == 1)
end

# Each attribute appears exactly once
for c in colors
    @constraint(model, sum(color[h, c] for h in houses) == 1)
end
for n in nationalities
    @constraint(model, sum(nationality[h, n] for h in houses) == 1)
end
for d in drinks
    @constraint(model, sum(drink[h, d] for h in houses) == 1)
end
for g in cigars
    @constraint(model, sum(cigar[h, g] for h in houses) == 1)
end
for p in pets
    @constraint(model, sum(pet[h, p] for h in houses) == 1)
end

# the brit lives in the red house
@constraint(model, [h in houses], nationality[h, :Brit] <= color[h, :Red])

# the Swede keeps dogs as pets
@constraint(model, [h in houses], nationality[h, :Swede] <= pet[h, :Dogs])

# the Dane drinks tea
@constraint(model, [h in houses], nationality[h, :Dane] <= drink[h, :Tea])

# Green is immediately left of White
for h in 1:4
    @constraint(model, color[h, :Green] <= color[h+1, :White])
end

# the owner of the Green house drinks coffee
@constraint(model, [h in houses], color[h, :Green] <= drink[h, :Coffee])

# the person who smokes Pall Mall rears birds
@constraint(model, [h in houses], cigar[h, :PallMall] <= pet[h, :Birds])

# The owner of the Yellow house smokes Dunhill
@constraint(model, [h in houses], color[h, :Yellow] <= cigar[h, :Dunhill])

```

```

# The man living in the centre house drinks milk
@constraint(model, drink[3, :Milk] == 1)

# The Norwegian lives in the first house
@constraint(model, nationality[1, :Norwegian] == 1)

# The man who smokes Blends lives next to the one who keeps cats
for h in houses
    if h > 1
        @constraint(model, cigar[h, :Blends] <= pet[h-1, :Cats] + pet[h+1, :Cats])
    elseif h == 1
        @constraint(model, cigar[h, :Blends] <= pet[h+1, :Cats])
    end
end

# The man who keeps horses lives next to the man who smokes Dunhill
for h in houses
    if h > 1
        @constraint(model, pet[h, :Horses] <= cigar[h-1, :Dunhill] + cigar[h+1, :Dunhill])
    elseif h == 1
        @constraint(model, pet[h, :Horses] <= cigar[h+1, :Dunhill])
    end
end

# The man who smokes Blue Master drinks beer
@constraint(model, [h in houses], cigar[h, :BlueMaster] <= drink[h, :Beer])

# The German smokes Prince
@constraint(model, [h in houses], nationality[h, :German] <= cigar[h, :Prince])

# The Norwegian lives next to the blue house
for h in houses
    if h > 1
        @constraint(model, nationality[h, :Norwegian] <= color[h-1, :Blue] + color[h+1, :Blue])
    elseif h == 1
        @constraint(model, nationality[h, :Norwegian] <= color[h+1, :Blue])
    end
end

# The man who smokes Blends has a neighbour who drinks water
for h in houses
    if h > 1
        @constraint(model, cigar[h, :Blends] <= drink[h-1, :Water] + drink[h+1, :Water])
    elseif h == 1
        @constraint(model, cigar[h, :Blends] <= drink[h+1, :Water])
    end
end

@objective(model, Min, 0)
set_silent(model)
optimize!(model)

```



```

# Check if the model solved
if termination_status(model) == MOI.OPTIMAL
    println("\nSolution found!\n")

    for h in houses
        println("House $h:")
        println("  Color: ", first(c for c in colors if value(color[h, c]) > 0.5))
        println("  Nationality: ", first(n for n in nationalities if value(nationality[h, n]) > 0.5))
        println("  Drink: ", first(d for d in drinks if value(drink[h, d]) > 0.5))
        println("  Cigar: ", first(g for g in cigars if value(cigar[h, g]) > 0.5))
        println("  Pet: ", first(p for p in pets if value(pet[h, p]) > 0.5))
        println("")
    end

    # Who owns the fish?
    for h in houses
        if value(pet[h, :Fish]) > 0.5
            owner = first(n for n in nationalities if value(nationality[h, n]) > 0.5)
            @printf("The owner of the fish is the %s.\n", owner)
        end
    end
else
    println("No solution found.")
end

```

Solution found!

House 1:

Color: Yellow
Nationality: Norwegian
Drink: Water
Cigar: Dunhill
Pet: Cats

House 2:

Color: Blue
Nationality: Dane
Drink: Tea
Cigar: Blends
Pet: Horses

House 3:

Color: Red
Nationality: Brit
Drink: Milk
Cigar: PallMall
Pet: Birds

House 4:

Color: Green
Nationality: German
Drink: Coffee
Cigar: Prince
Pet: Fish

House 5:

Color: White
Nationality: Swede
Drink: Beer
Cigar: BlueMaster
Pet: Dogs

The owner of the fish is the German.

In []: